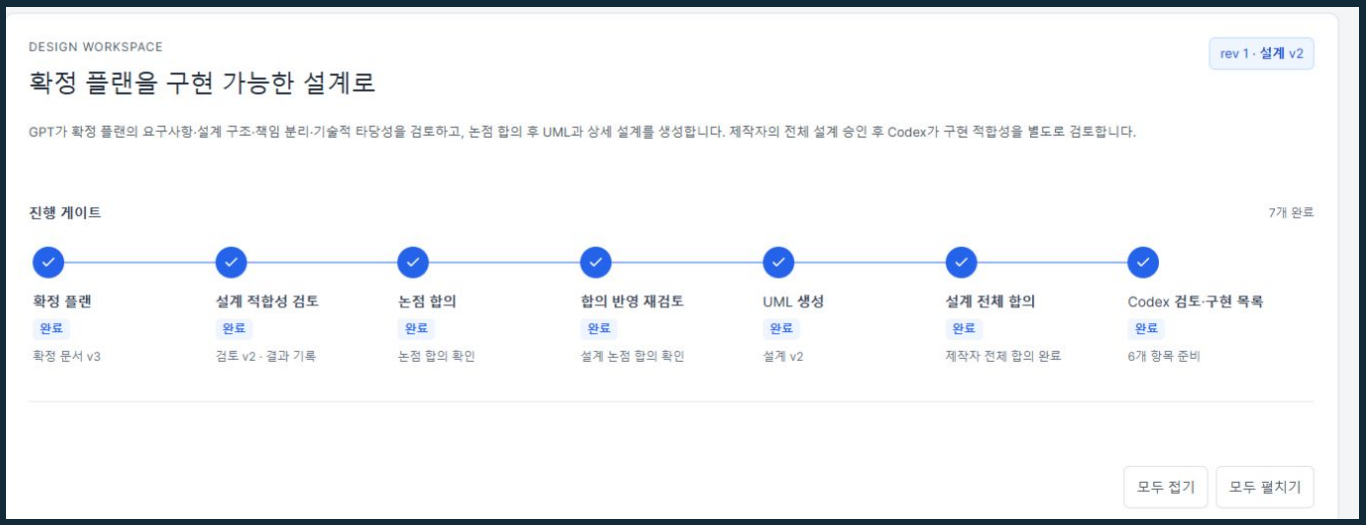
Agent Workflow

게임 제작 의도를 실제 결과로 연결하는 시스템

제작 공정 설계 · 개발 도구 · Unreal 콘텐츠 통합 · 검증



SF Team Hub에서 실제 운영하는 설계·합의·구현 인계 공정

Creator가 의도와 품질을 판단하고,
Agent가 구현과 검증을 이어가도록 제작 기반을 설계했다.

PORTFOLIO / 2026.10

OVERVIEW

제작의 판단과 실행을 하나의 흐름으로 연결하다

ProjectSF는 Unreal Engine 기반 게임 프로젝트이자, 사람과 AI Agent가 같은 결과물을 이어 만드는 제작 공정의 실험 환경이다. 이 포트폴리오는 제작 공정·도구·콘텐츠를 연결하면서 해결한 기술 문제와 실제 결과를 담는다.

Creator
목표 · 설계 · 결과 수용

→

AI Agent
구현 · 실행 · 기본 검증

→

Programmer
구조 검토 · 공정 개선

해결하려는 문제

대화·코드·에디터·검증 기록이 분리되면 현재 결정과 다음 작업을 다시 맞추는 비용이 생긴다. 같은 목표를 세션과 도구가 바뀌어도 이어갈 수 있는 구조가 필요했다.

제작 공정 설계

Issue·Spec·직접 요청을 작업 규모에 맞춰 사용하고, 버전·결정·검증 상태의 원본을 분리했다.

도구와 제품의 연결

서버 MCP·Unreal Toolset·DataForge·RPCNet을 실제 제작과 통합 검증의 경로로 연결했다.

설계 원칙

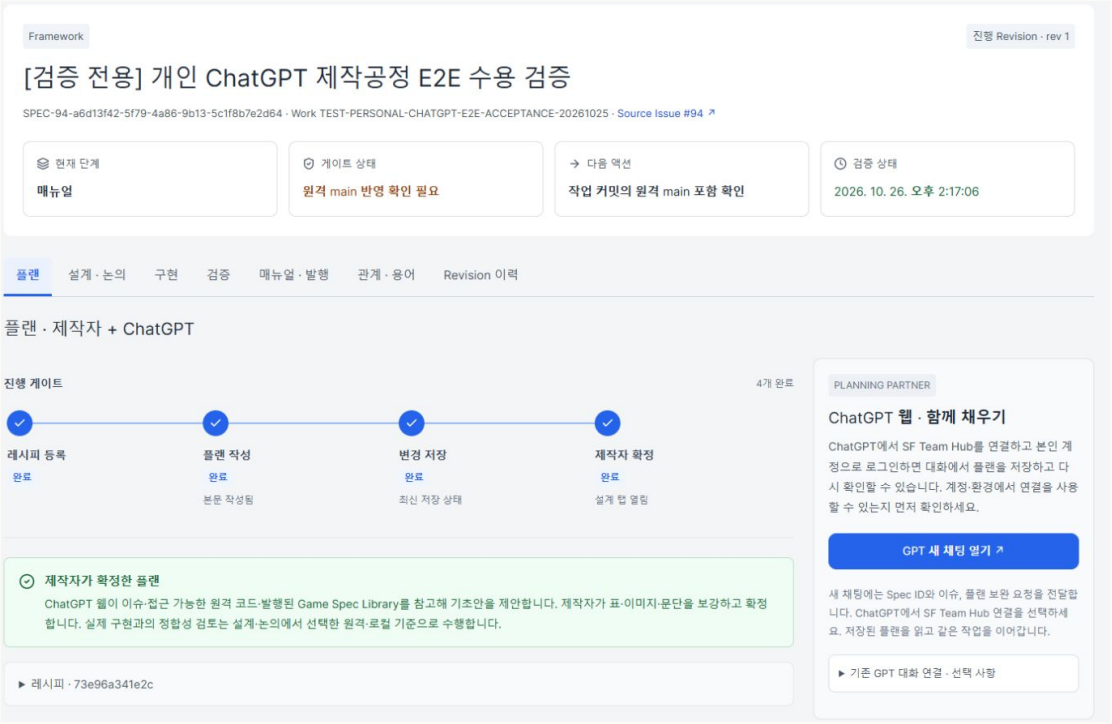
공식 엔진 기능과 기존 구조를 먼저 사용한다. 도구 실행 성공, 실제 제품 동작, 사람의 수용을 각각 판단한다.

대표 기술: Unreal Engine 5.8 / C++ / GAS / Python / C# / TypeScript / MCP

CASE 01 / SF TEAM HUB

채팅이 끝나도 제작은 이어지게

기획·설계·구현·검증의 현재 상태를 복원할 수 있는 협업 시스템을 설계했다.



실제 운영 화면 · Spec94의 문서와 현재 단계, 플랜 작성·확정 상태

문제

Issue 본문과 대화만으로 복합 작업을 진행하면 어떤 문서와 설계가 합의된 기준인지, 무엇이 실제로 완료됐는지 구분하기 어려웠다.

설계 판단

Spec Store가 문서·Revision·결정·Evidence를 소유하고 업무보드는 그 상태를 보여주도록 했다. GPT와 Codex는 공통 서버 MCP로 같은 Spec을 읽고 저장·재조회한다.

결과

새 대화에서 Spec ID로 상태를 복원하고, 저장 결과를 독립적으로 대조했다. 승인된 두 계정의 Android 공정을 확인했으며 팀원별 플러그인 배포는 채택하지 않았다.

CASE 01 / DESIGN TO EXECUTION

설계와 구현 사이의 인계 기준을 구조화하다

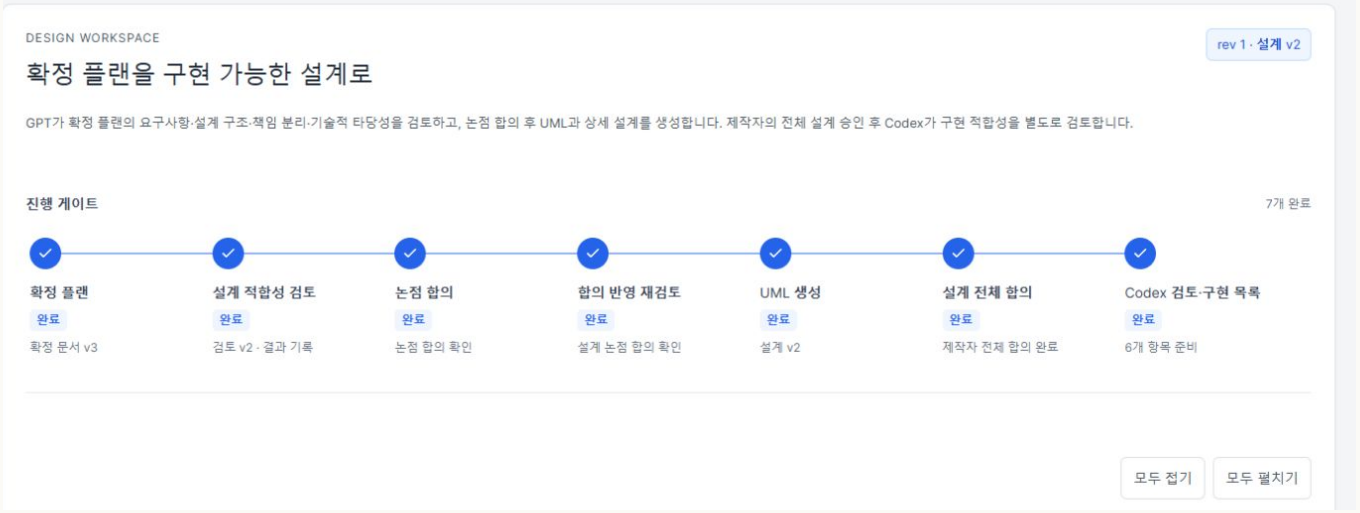
플랜
목표 · 범위 · 완료 조건

→

합의된 설계
논점 · UML · 버전

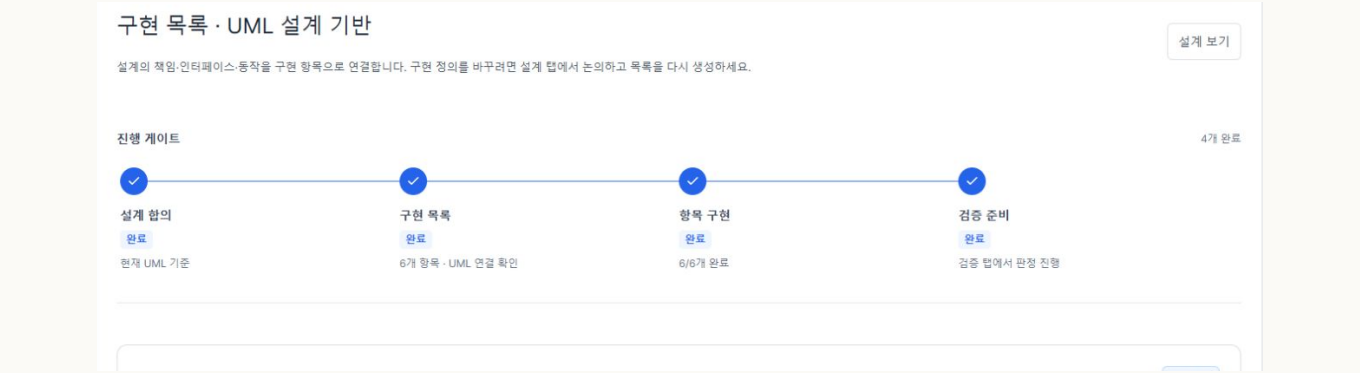
→

구현 목록
변경 · 의존성 · 검증



확정 플랜에서 설계 검토·논점 합의·UML·Codex 인계로 이어지는 실제 화면

설계는 도식 생성에서 끝나지 않는다. 각 구현 항목이 어떤 설계의 책임과 동작을 구현하는지 연결하고, 설계 변경 시 영향을 받는 항목을 다시 검토하도록 했다.

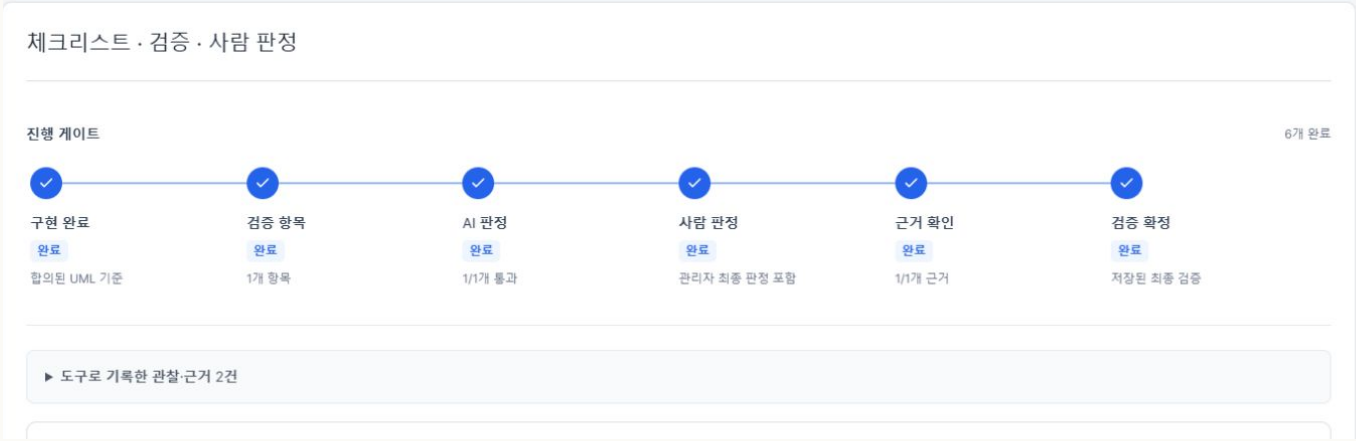


실제 구현 탭 · UML에 연결된 작업 목록과 항목별 완료 상태

GPT는 기획·설계 협업, Codex는 실제 저장소 조사와 구현 적합성 검토·제작을 맡는다. 사용자 결정과 Agent의 기술 판단을 서로 대신하지 않는 인계 구조다.

CASE 01 / QUALITY & RESULTS

완료의 근거를 확인할 수 있는 제작 공정



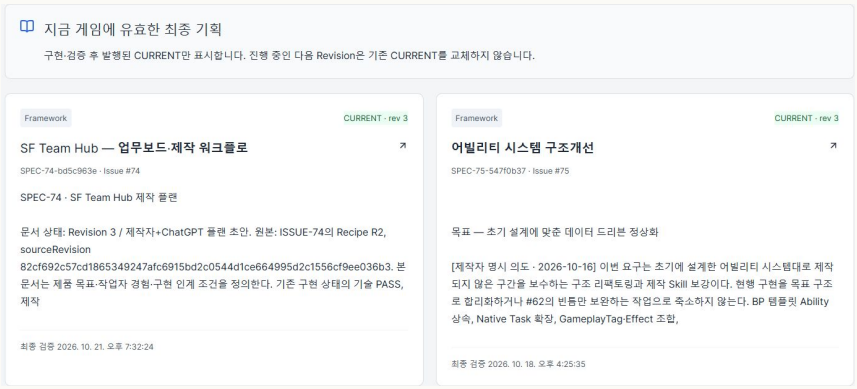
실제 검증 화면 · 구현, 검증 정의, AI·사람 판정, 근거 확인을 연결

같은 PASS라도 의미가 다르다. 구현 결과·검증 증거·사람 수락·발행을 별도 상태로 관리해, 확인하지 않은 품질을 완료로 표시하는 문제를 줄이는 구조를 만들었다.

20
SPEC-74 R3 구현 항목 완료

18
같은 발행 시점의 기술 PASS

6
SPEC-94 승인 범위 VERIFY



Game Spec Library · 발행된 CURRENT를 다음 제작의 기준으로 재사용

SPEC-74 R3는 사람 수락과 CURRENT 발행을 기록했다. SPEC-94의 6개 항목은 별도 검증 범위이며 iOS·원격 main·발행의 남은 조건은 마지막 자료 기준에 명시했다.

CASE 02 / WORLD PRODUCTION

지형·도로·수계를 반복 수정 가능한 월드로

GIS 입력과 Unreal 제작 도구를 연결하고, 생성 후에도 기존 결과를 보존하며 수정하도록 했다.



NeoSeoul 프로토타입 · 2026-10-07 저장 상태의 지형·수계 기반



대표 S001 구간 / 2026-10-08 · 기존 기반 도로면 위에 차선·양측 보도를 연결

입력의 좌표·단위·feature identity를 유지하고, 기반 도로면과 차선·보도, 지형과 수계를 분리했다. 변경 영역의 저장·재로드와 비대상 보존을 함께 확인했다.

247
표시·보도 구간 제작

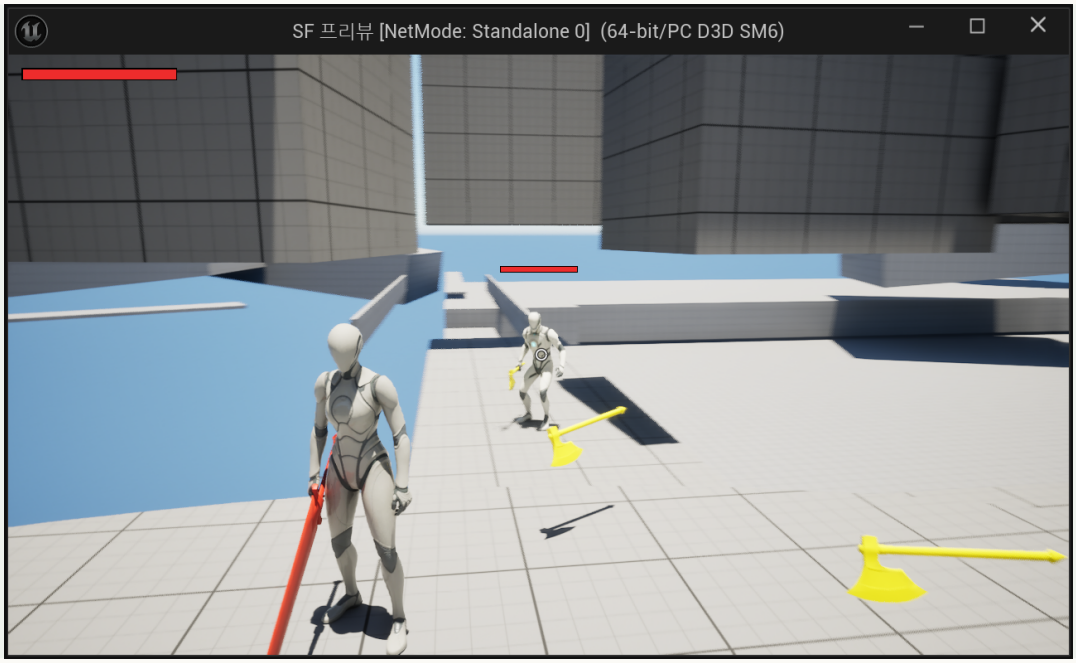
32
신규 기반 도로면

14
기존 기반 도로면 보존

#54 당시 제작 기록. 대표 화면 한 쌍을 전체 구간의 품질 근거로 확대하지 않는다.

CASE 03 / COMBAT & CHARACTER DATA

전투 규칙을
편집 가능한 데이터와 실행으로



Teyvat Standalone PIE / 2026-10-10 · 락온과 플레이어·몬스터 체력 UI

입력, 공격 타이밍, 판정 위치, AI 선택을 한 곳에 몰아넣으면 작은 조정도 여러 에셋의 중복 수정으로 이어진다. 편집 지점과 runtime 책임을 나누는 방식으로 전투 기반을 정리했다.

프로필 데이터
이동 · 액션 · AI · 쿨타임

→

Montage / Notify
공격창 · 보호 · 전환 시점

→

BP / GAS
본·소켓 · 상태 · 실제 피해

구현과 확인

3타·가드·취소·접촉 판정과 캐릭터 프로필을 연결했다. 실제 PIE에서 피해·행동 복귀·저장 결과를 대조하고 기존 Hero 동작의 회귀를 확인했다.

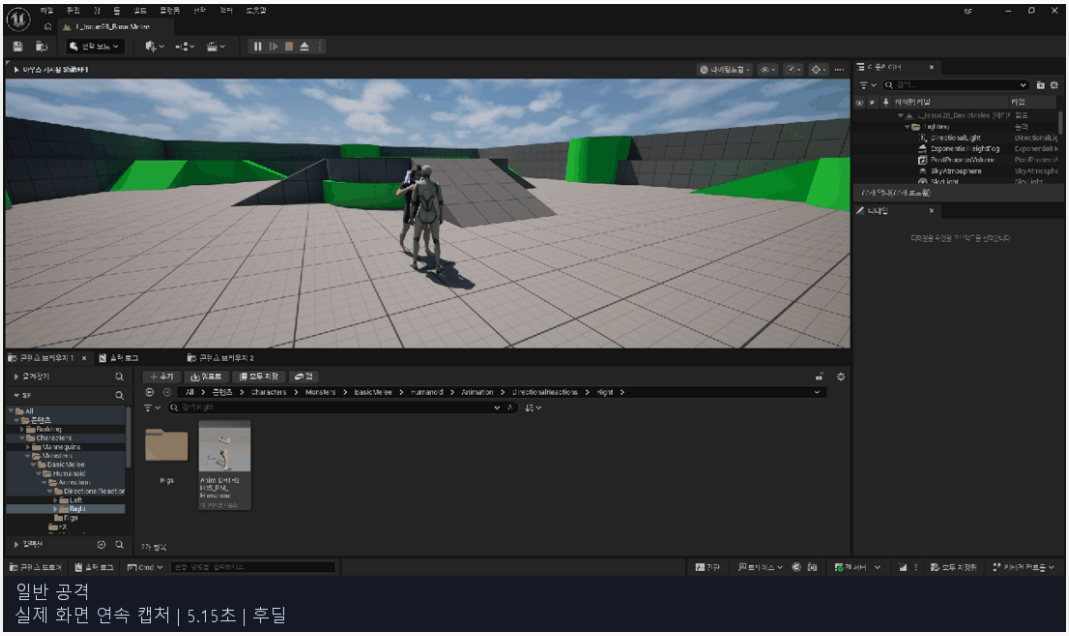
25/25
프로필·전투 자동 검사 PASS

9개체
그룹 PIE 관측

2026-10-09 #62 checkpoint. 최종 전투 체감·네트워크·패키징 전체 수용과는 구분한다.

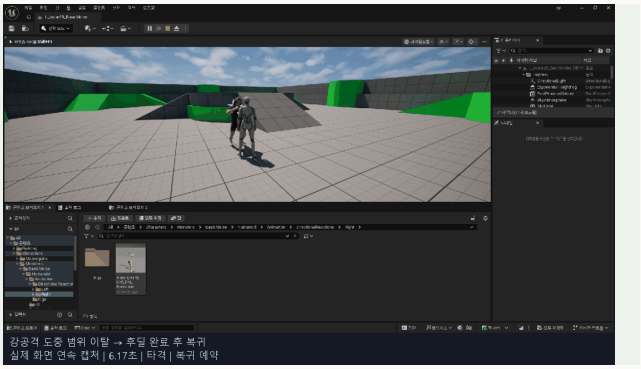
CASE 04 / METAHUMAN INTEGRATION

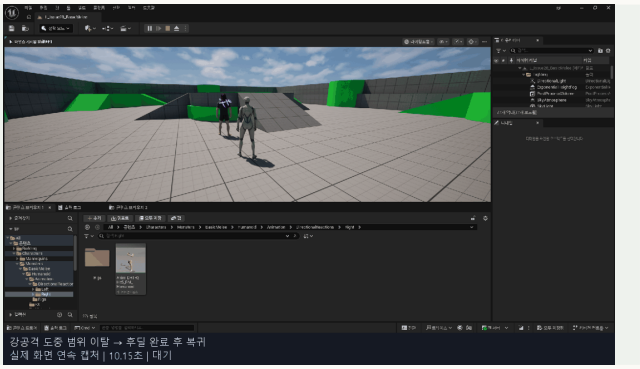
제작 원본을 유지하며 외형과 게임 동작을 통합하다



실제 기본 몬스터 일반 공격 캡처 / 2026-10-06

MetaHuman의 편집 가능한 원본과 조립 BP를 유지하면서, Body·Face·Clothing의 애니메이션 참조를 게임 캐릭터 기반에 연결했다. 외형 통합과 몬스터 행동을 각각 확인했다.





같은 원본 캡처의 강공격 완료와 대기 복귀 장면

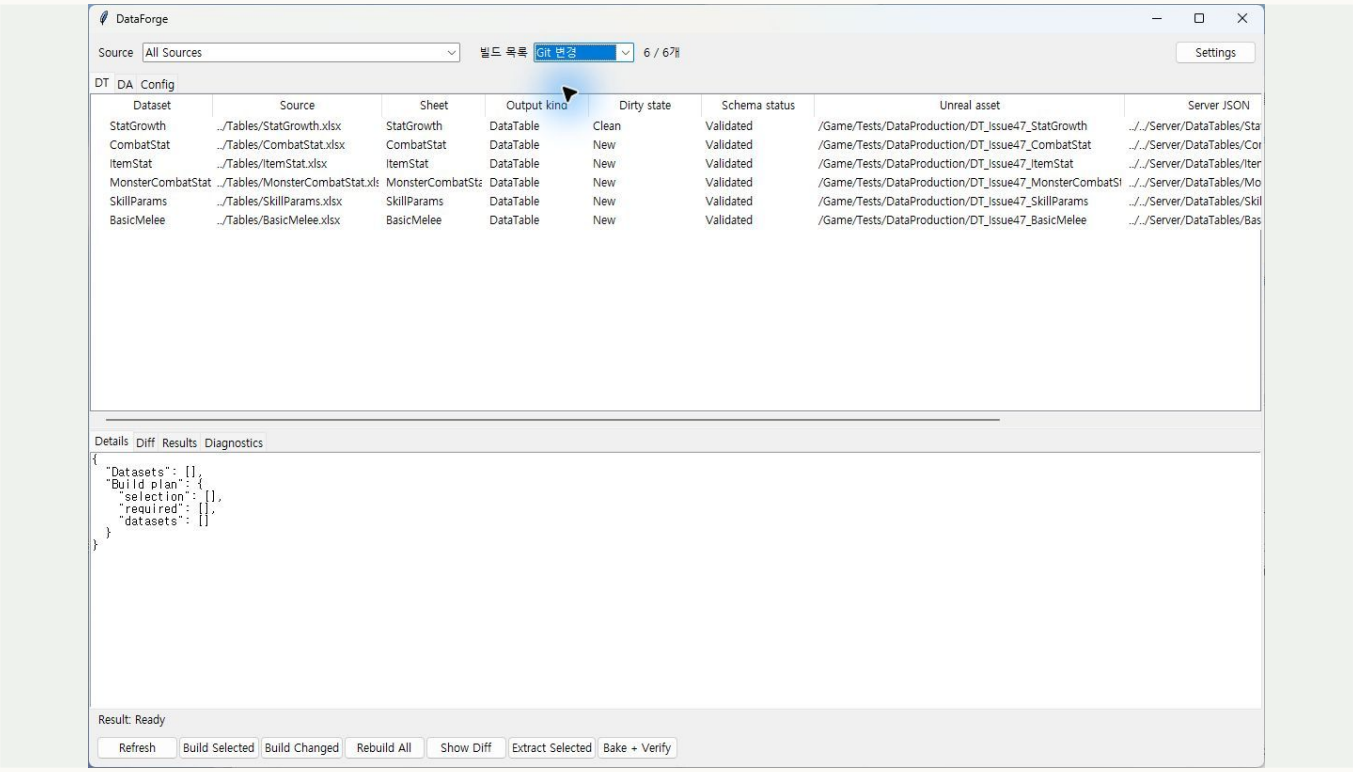
기술적 결과

실제 PIE에서 mesh·AnimInstance·Leader Pose를 조회했다. main 통합 후 자동 검사 9건에서 실패0, MapCheck 오류·경고0을 기록했다.

원본 GIF의 실제 프레임. 최종 외형·의상·미세 동작 품질과 멀티플레이 수용을 대신하지 않는다.

CASE 05 / DATAFORGE

Excel에서 실제 게임 데이터까지 하나의 의미로 연결하다



실제 DataForge GUI / 2026-10-07 · dataset 상태와 변경 대상, 빌드·검증 결과를 확인

Client와 Server 데이터를 따로 변환하면 같은 기획 값이 서로 다른 결과가 될 수 있다. GUI와 CLI가 공통 Core를 사용하고, 하나의 정규화 결과에서 두 산출물을 만들도록 설계했다.

Excel / Schema
기획자가 편집하는 원본

→

공통 Core
정규화 · 검증 · 변경 판정

→

UE / Server
Bake · Readback · Parity

설계의 핵심

파일 생성에서 멈추지 않고 별도 프로세스가 읽은 실제 UDataTable 값을 대조한다. 잘못된 입력은 마지막 정상 산출물을 보존하고, 의미가 같은 변경은 불필요한 재생성을 피한다.

DataAsset·Config는 Unreal 원본을 유지하며 native 조회·추출. DA·Config write-back은 범위에 포함하지 않는다.

CASE 06 / RPCNET

통신 기반과 게임의 의미를 분리한 네트워크 프레임워크

상용 Unreal 프로젝트에서의 클라이언트 네트워크 전환·프레임워크 설계 경험을 바탕으로, ProjectSF에서 별도로 개발한 개인 R&D;다. 실제 서버와 Unreal Client를 연결해 재사용 경계를 검증했다.

RPCNet Runtime

Session · Transport · Codec · Envelope

RPCNet Toolchain

ProtocolGenerator · 생성 계약 · 설치 도구

ProjectSF Consumer

Schema · 최종 Endpoint · 게임별 Domain

Unreal Client / Bot

실제 TCP와 패킷 처리

→

Gateway / AS

라우팅 · 인증 · 입장권

→

GS / Repository

영속 Player 상태

생성 코드의 소유권

Schema에서 DTO·Proxy·Stub을 생성하고, 직접 작성한 Endpoint와 Domain Service로 연결한다. 재생성으로 게임 로직을 덮어쓰지 않는다.

검증의 결과

Mock/Memory 검사와 실제 서버 상호운용성을 분리했다. 당시 RPCNet checkpoint 37개 PASS와 실제 Unreal Client 통신을 확인했다. Gateway·AS·GS는 이 통신 검증을 위해 구축했다.

상용 코드와 개인 구현은 분리. 독립 상용 서버 출시나 WAN·운영 HA 성과로 소개하지 않는다.

ENGINEERING APPROACH

제작 사례를 재사용 가능한 기반으로 남기다

개별 기능을 만드는 것과 다음 기능을 안전하게 만들 수 있는 구조를 남기는 것은 함께 다뤄야 한다. ProjectSF에서는 여러 제작 영역에 같은 설계 기준을 적용했다.

01 원본과 생성 결과의 소유권 분리

DataForge는 authoring 데이터와 산출물을, RPCNet은 생성 코드와 사람이 작성한 로직을 구분한다. 월드 제작은 생성 owner와 수작업 영역을 나눈다.

DataForge / RPCNet / World Production

02 엔진·플랫폼의 기존 계약 재사용

Unreal이 해석해야 할 데이터는 엔진을 통해 읽고, 실제 제작 효과는 공식 MCP 계약과 대조한다. 반복 필요가 확인된 부분에 도메인 Toolset을 더한다.

Official Unreal MCP / Native Readback

03 결과를 재조회할 수 있는 검증

저장 receipt만으로 성공을 선언하지 않는다. 실제 문서·UDataTable·에셋·runtime 상태를 다시 읽고, 사람이 판단해야 할 품질과 기술 검사를 구분한다.

SF Team Hub / Bake & Parity / PIE

다음 제작의 재사용성을 높이는 역할

Creator의 목표와 실제 결과를 연결하고, 완료 작업들을 종합 검토해 구조·도구·Workflow·Skill·Test를 개선하는 Production Engineering을 지향한다.

PROJECT & REFERENCES

프로젝트 정보와 자료

ProjectSF의 현재 원본과 공개 포트폴리오에서 사례별 구현·검증 근거를 확인할 수 있다.

프로젝트

Unreal Engine 기반 개인 게임 프로젝트 ProjectSF. 게임 제작과 함께 사람·Agent 협업 공정, 개발 도구와 통합 검증을 연구한다.

검증 범위

월드·전투·MetaHuman 이미지는 2026년 10월의 해당 프로토타입 기록이다. 신규 월드 완성·최종 시각 수용·전체 패키징을 뜻하지 않는다.

현재 공정

공통 서버 MCP의 실제 저장·readback과 두 계정 Android 공정을 확인했다. Spec94는 승인된 6개 항목의 VERIFY를 확정했으며 iOS는 보류, 원격 main 확인·CURRENT 발행은 별도다. 국소 회귀·빌드 PASS와 전체 TypeScript 기존 오류3건 FAIL을 함께 보존한다.

공개 포트폴리오 ↗

실제 사이트 공정과 화면 ↗

구현·검증 현황 ↗

데이터 제작 사례 ↗

RPCNet과 서버 통신 검증 ↗

원본 대조·공개 범위 기록 ↗

2026.10.26 기준. 효율 향상률·전체 비용 절감은 별도로 계측할 범위다. 코드·게임·서버 테스트는 PDF 편집 과정에서 재실행하지 않았다.